

CL-2006 Operating System Lab

Code Explanation Guide

Lab 05: Process Management

fork() · exec() · Zombie & Orphan Processes

*Complete Line-by-Line Code Walkthrough
with Theoretical Concepts & Quiz Preparation*

7 Examples Fully Explained:

- Example 1: Basic fork() with Process IDs
- Example 2: Separate Address Spaces
- Example 3: Creating Multiple Child Processes
- Example 4: Creating a Zombie Process
- Example 5: Creating an Orphan Process
- Example 6: Simple exec()
- Example 7: fork() + exec() Together (Mini Shell)

Contents

1	Key Theoretical Concepts – Lab 05	2
1.1	What is a Process?	2
1.2	Process ID (PID)	2
1.3	What is a System Call?	2
1.4	System Calls Quick Reference	3
1.5	fork() Return Values – MUST MEMORIZE	3
1.6	Zombie vs Orphan – Side by Side	3
2	Example 1: Basic fork() with Process IDs	4
2.1	Line-by-Line Explanation	4
3	Example 2: Separate Address Spaces	7
3.1	Line-by-Line Explanation	8
4	Example 3: Creating Multiple Child Processes	10
4.1	Line-by-Line Explanation	11
5	Example 4: Creating a Zombie Process	13
5.1	Line-by-Line Explanation	13
5.2	The Fix: Using wait()	14
6	Example 5: Creating an Orphan Process	16
6.1	Line-by-Line Explanation	16
7	Example 6: Simple exec()	18
7.1	Line-by-Line Explanation	18
8	Example 7: fork() + exec() Together – Mini Shell	20
8.1	Line-by-Line Explanation	21
9	Quick Reference – Quiz Cheat Sheet	23
9.1	fork() Rules – Must Know	23
9.2	exec() Rules – Must Know	23
9.3	Zombie & Orphan Rules	23
9.4	Compilation	23

Key Theoretical Concepts – Lab 05

What is a Process?

OS Concept – Process vs Program

A **program** is a passive file sitting on disk (e.g., `/bin/ls`). A **process** is a program that is currently loaded into memory and executing. When you run `./myprogram`, the OS creates a process from that program.

Each process has its own: **address space** (code + data + stack), **program counter** (which instruction to run next), **registers** (CPU state), and **PID** (unique identifier). The OS tracks all this in a **Process Control Block (PCB)**.

Process ID (PID)

Every running process has a unique **PID (Process Identification Number)**. Key PIDs to know:

PID	Process	Significance
1	<code>init</code> / <code>systemd</code>	First process started by the kernel. Adopts orphan processes.
> 1	User processes	Assigned sequentially. Reused when a process terminates.

What is a System Call?

OS Concept – System Calls

A **system call** is the mechanism by which a program asks the OS kernel to do something privileged on its behalf. Your C program runs in **user mode** (restricted). When it calls `fork()`, `read()`, or `write()`, execution temporarily switches to **kernel mode** where the OS performs the requested action, then switches back.

Think of it as: your program is a customer, the kernel is the bank, and a system call is filling out a request form.

System Calls Quick Reference

System Call	Header	Purpose
fork()	<unistd.h>	Create a new child process (copy of parent)
getpid()	<unistd.h>	Get PID of the calling process
getppid()	<unistd.h>	Get PID of the parent process
wait(NULL)	<sys/wait.h>	Parent waits for any child to finish
execl()	<unistd.h>	Replace process with new program (by path)
execvp()	<unistd.h>	Replace process with new program (searches PATH)
exit(status)	<stdlib.h>	Terminate current process with a status code
sleep(n)	<unistd.h>	Pause execution for n seconds

fork() Return Values – MUST MEMORIZE

Return Value	Meaning	Which Process?
> 0 (positive)	The PID of the child	Parent process
0	This is the child	Child process
-1	fork() failed	Error – no child created

Most Important fork() Rule

After `fork()`, **both** parent and child continue executing from the line immediately after the `fork()` call. The only way to tell them apart is by checking the return value:

- if (`pid == 0`) $\Rightarrow$ you are in the **child**
- if (`pid > 0`) $\Rightarrow$ you are in the **parent**
- if (`pid < 0`) $\Rightarrow$ `fork()` **failed**

Zombie vs Orphan – Side by Side

	Zombie Process	Orphan Process
Definition	Child finished but parent never called <code>wait()</code>	Parent exits first while child is still running
Who exits first?	Child	Parent
Problem	Wastes a slot in process table	None really – just gets re-parented
Solution	Parent must call <code>wait()</code>	<code>init</code> (PID 1) automatically adopts orphan
Visible as	<defunct> in <code>ps</code> output	Child's <code>getppid()</code> returns 1

Example 1: Basic fork() with Process IDs

What This Example Teaches

The most fundamental fork() program. Creates one child process and demonstrates that both parent and child run after fork(), but the code BEFORE fork() runs only once (in the parent before fork is called).

```

1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 int main() {
7     printf("Before Checking\n");
8     printf("Creating child process\n");
9
10    int i = fork();
11    printf("fork() returned: %d\n", i);
12
13    if (i == 0) {
14        printf("I am a CHILD Process pid = %d, ppid = %d\n",
15              getpid(), getppid());
16    }
17    else if (i > 0) {
18        printf("I am a PARENT Process pid = %d, ppid = %d\n",
19              getpid(), getppid());
20    }
21    else {
22        perror("fork failed");
23        return 1;
24    }
25
26    printf("After forking\n");
27    return 0;
28 }
```

Listing 1: basic_fork.c – Basic fork() with Process IDs

Line-by-Line Explanation

```
#include <stdio.h>
```

Standard I/O library. Provides printf() and perror().

```
#include <sys/types.h>
```

Provides the pid_t data type (used for process IDs). Best practice to include whenever using fork().

```
#include <unistd.h>
```

Provides POSIX system calls: `fork()`, `getpid()`, `getppid()`, `sleep()`, `execl()`. Required for all process-related programming.

```
#include <stdlib.h>
```

Provides `exit()`, `malloc()`, and other general utilities.

```
printf("Before Checking\n"); printf("Creating child process\n");
```

These two lines execute **only once** – in the parent process, before `fork()` is called. At this point, only one process exists. This is why “Before Checking” appears only once in the output.

```
int i = fork();
```

This is the key line. The OS creates a child process that is an **exact copy** of the parent.

– In the **parent**: `i` gets the child’s PID (e.g., 3181)

– In the **child**: `i` gets 0

After this line, TWO processes exist and both continue executing from the next line.

```
printf("fork() returned:  %d\n", i);
```

This line runs in **BOTH** processes. Parent prints the child’s PID (e.g., 3181). Child prints 0. That’s why you see this line twice in the output with different values.

```
if (i == 0) {
```

This condition is **true only in the child** (fork returned 0 to the child). Only the child process enters this block.

```
printf("I am a CHILD Process pid = %d, ppid = %d\n", getpid(),  
getppid());
```

Child prints:

`getpid()` – the child’s own PID (e.g., 3181)

`getppid()` – the parent’s PID (e.g., 3180). Shows the parent-child relationship.

```
else if (i > 0) {
```

This condition is **true only in the parent** (fork returned the child’s PID, which is > 0). Only the parent process enters this block.

```
printf("I am a PARENT Process pid = %d, ppid = %d\n", getpid(),  
getppid());
```

Parent prints:

`getpid()` – parent’s own PID (e.g., 3180)

`getppid()` – the shell’s PID (e.g., 3144). The shell is the parent of the parent.

```
else { perror("fork failed"); return 1; }
```

If `fork()` returns -1, it failed. `perror()` prints the system error message. Common causes: system process limit reached, not enough memory.

```
printf("After forking\n");
```

This line is **outside all if/else blocks**. It runs in **BOTH parent and child**. This is why “After forking” appears **twice** in the output.

Expected Output

```
Before Checking          <-- once (parent only, before fork)
Creating child process   <-- once (parent only, before fork)
fork() returned: 3181    <-- parent (got child's PID)
I am a PARENT Process pid = 3180, ppid = 3144
After forking            <-- parent
fork() returned: 0       <-- child (got 0)
I am a CHILD Process  pid = 3181, ppid = 3180
After forking            <-- child
```

Key Insight

Code BEFORE `fork()` runs **once**. Code AFTER `fork()` runs **twice** (once in each process). The `if/else` on the return value is how you give different behavior to parent and child.

Example 2: Separate Address Spaces

What This Example Teaches

Proves that after fork(), parent and child have **completely separate copies** of all variables. A change in one process does NOT affect the other – even for global variables.

```

1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 void parent_process(int pvar);
6 void child_process(int cvar);
7
8 int y = 10;
9
10 int main() {
11     int x = 0;
12     printf("Before forking\n");
13     printf("Creating child process\n");
14
15     int i = fork();
16
17     if (i == 0) {
18         child_process(x);
19     } else {
20         parent_process(x);
21     }
22
23     printf("After forking\n");
24     return 0;
25 }
26
27 void child_process(int a) {
28     y += 2;
29     a = 3;
30     printf("Child:  a = %d, y = %d\n", a, y);
31 }
32
33 void parent_process(int b) {
34     b = 2;
35     y += 5;
36     printf("Parent: b = %d, y = %d\n", b, y);
37 }

```

Listing 2: separate_addr.c – Separate address spaces after fork()

Line-by-Line Explanation

```
void parent_process(int pvar); void child_process(int cvar);
```

Function prototypes declared before `main()` because these functions are defined AFTER `main()`. Without these prototypes, the compiler would not know they exist when `main()` tries to call them.

```
int y = 10;
```

A **global variable** declared outside any function. Global variables are stored in the data segment of the process and are shared across all functions within the same process. After `fork()`, **each process gets its own independent copy** of this global variable, both starting at 10.

```
int x = 0;
```

A **local variable** on the stack of `main()`. Also copied to both processes after `fork()`. Both parent and child start with `x = 0`.

```
if (i == 0) { child_process(x); } else { parent_process(x); }
```

After `fork()`, each process goes to its own function. Both functions receive `x` (value 0) as an argument. Each function gets its own parameter copy on its own stack.

```
void child_process(int a) {
```

Child's function. Parameter `a` starts as 0 (copy of `x`).

```
y += 2;
```

Child modifies **its own copy** of the global `y`: $10 + 2 = 12$. The parent's `y` is still 10 at this moment (separate address spaces).

```
a = 3;
```

Child sets its local parameter to 3. This does not affect `x` in `main()` because function parameters are copies (pass by value in C).

```
printf("Child:  a = %d, y = %d\n", a, y);
```

Prints 3 and 12 – the child's modified values.

```
void parent_process(int b) {
```

Parent's function. Parameter `b` starts as 0 (copy of `x`).

```
b = 2; y += 5;
```

Parent sets `b=2` and modifies **its own copy** of `y`: $10 + 5 = 15$. The child's `y` stays at 12.

```
printf("Parent:  b = %d, y = %d\n", b, y);
```

Prints 2 and 15 – the parent's modified values.

Expected Output

Before forking

Creating child process

Parent: b = 2, y = 15 <-- parent's OWN copy of y

After forking

Child: a = 3, y = 12 <-- child's OWN copy of y (different!)

After forking

Proof of Separate Address Spaces

Both parent and child started with y = 10. Parent added 5 (got 15), child added 2 (got 12). Neither change affected the other. This is the fundamental proof that fork() gives each process its own independent copy of all memory.

Example 3: Creating Multiple Child Processes

What This Example Teaches

How multiple `fork()` calls create a **tree of processes**. Each `fork()` doubles the number of processes. Understanding which process is running which part of the code is the key skill here.

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("node:1, pid = %d, ppid = %d\n", getpid(), getppid())
6         ;
7
8     int i = fork();
9
10    if (i == 0) {
11        printf("node:3, pid = %d, ppid = %d\n", getpid(),
12            getppid());
13    }
14    else {
15        i = fork();
16        if (i == 0) {
17            printf("node:2, pid = %d, ppid = %d\n", getpid(),
18                getppid());
19
20            i = fork();
21            if (i == 0) {
22                printf("node:6, pid = %d, ppid = %d\n", getpid()
23                    , getppid());
24            } else {
25                i = fork();
26                if (i == 0) {
27                    printf("node:5, pid = %d, ppid = %d\n",
28                        getpid(), getppid());
29                }
30            }
31        }
32    }
33
34    return 0;
35 }

```

Listing 3: multi_fork.c – Creating multiple child processes (process tree)

Line-by-Line Explanation

```
printf("node:1, pid = %d, ppid = %d\n", getpid(), getppid());
```

Runs **only once** – before any fork(). This is the original process, labeled node 1 in the process tree.

```
int i = fork();
```

First fork(). Creates the first child.

- **Parent** (original process): `i > 0`, goes into `else` block
- **Child 1** (new process): `i == 0`, goes into `if` block

```
if (i == 0) { printf("node:3, ..."); }
```

Only **Child 1** enters here. It prints “node:3” and then falls through to `return 0`. Child 1 is done – it becomes node 3.

```
else { i = fork();
```

Only the **original parent** (node 1) enters here. It calls fork() again, creating a second child.

- **Original parent** (node 1): `i > 0`, skips inner `if`
- **Child 2** (new process): `i == 0`, enters inner `if`

```
if (i == 0) { printf("node:2, ..."); }
```

Only **Child 2** enters here. It prints “node:2” – this is node 2 in the tree. Child 2 then continues executing INSIDE this block.

```
i = fork();
```

Called **inside** the `if (i == 0)` block – so only **node 2** calls this. Creates node 2’s first child.

- **Node 2**: `i > 0`, goes into `else`
- **Child of node 2** (first): `i == 0`, goes into inner `if`

```
if (i == 0) { printf("node:6, ..."); }
```

Node 2’s first child: prints “node:6”. This is the deepest branch.

```
else { i = fork(); if (i == 0) { printf("node:5, ..."); } }
```

Node 2 (parent of node 6) calls fork() AGAIN. Creates node 5.

- **Node 2**: `i > 0`, skips inner `if`
- **Node 5** (new child): `i == 0`, prints “node:5”

Process Tree Visualization

```
node:1 (original)
|-- node:3 (first fork, child branch)
|-- node:2 (second fork from node:1)
    |-- node:6 (first fork from node:2)
    |-- node:5 (second fork from node:2)
```

Verify: node:3's ppid = node:1's pid. node:2's ppid = node:1's pid. node:5 and node:6's ppid = node:2's pid.

OS Concept – Tracing fork() Through Nested Ifs

The critical rule: when a process calls fork(), **only that process's child** gets `i = 0` and enters the inner if. The ORIGINAL parent from the outer fork gets `i > 0` and does NOT enter the inner if. This is what shapes the tree structure.

Example 4: Creating a Zombie Process

What This Example Teaches

Demonstrates what happens when a child exits but the parent never calls `wait()`. The child becomes a **zombie** – dead but still visible in the process table as `<defunct>`.

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 #include <stdlib.h>
5
6 void create_zombie() {
7     if (fork() == 0) {
8         printf("Terminating Child, PID = %d\n", getpid());
9         exit(0);
10    }
11    else {
12        printf("Running Parent, PID = %d\n", getpid());
13        while (1);
14    }
15 }
16
17 int main() {
18     printf("Hello from main\n");
19     create_zombie();
20     return 0;
21 }
```

Listing 4: zombie.c – Creating and observing a zombie process

Line-by-Line Explanation

```
void create_zombie() {
```

A helper function that contains the fork logic. Called from `main()`. The zombie creation happens inside this function.

```
if (fork() == 0) {
```

Calls `fork()` and immediately checks if the return value is 0.

If true: we are in the child process $\Rightarrow$ enters this block

If false (else): we are in the parent $\Rightarrow$ goes to else

Note: this is a compact way to write the fork check (no storing return value first).

```
printf("Terminating Child, PID = %d\n", getpid());
```

Child announces its PID. Remember this number – you will look for it in `ps` output.

```
exit(0);
```

Child **terminates immediately** with exit status 0. At this point, the child is technically done. But the OS cannot fully remove the child from the process table until the parent calls `wait()` to collect the exit status. So the child becomes a **zombie** (<defunct> in `ps`).

```
else {
```

Parent block. The parent did NOT get 0 from `fork()`, so it goes here.

```
printf("Running Parent, PID = %d\n", getpid());
```

Parent prints its PID so you can kill it later if needed.

```
while (1);
```

Parent runs an infinite loop and **NEVER** calls `wait()`. This is the bug. Since the parent never collects the child's exit status, the zombie persists in the process table indefinitely.

How to Observe the Zombie

```
# Terminal 1: Run the program
```

```
$ gcc zombie.c -o zombie && ./zombie
```

```
# Terminal 2: Look for the zombie
```

```
$ ps -al
```

```
(Look for <defunct> next to a process -- that's the zombie)
```

```
# Kill the parent to clean up
```

```
$ kill <parent-pid>
```

The Fix: Using `wait()`

```
1  /* BAD -- creates zombie */
2  int pid = fork();
3  if (pid > 0) {
4      sleep(10);          /* parent sleeps -- child is zombie
5                          here */
6      printf("Parent\n");
7  }
8
9  /* GOOD -- no zombie */
10 int pid = fork();
11 if (pid > 0) {
12     wait(NULL);          /* parent WAITS for child to finish */
13     printf("Parent\n"); /* child is properly cleaned up now */
14 }
```

Listing 5: Without `wait()` (BAD – creates zombie) vs With `wait()` (GOOD)

```
wait(NULL);
```

Tells the parent to **block** (pause) until any child process finishes. When the child exits, `wait()` collects the child's exit status and removes the zombie from the process table. `NULL` means we don't care about the exit status value – we just want to reap the zombie.

OS Concept – Why Zombies Are a Problem

Each zombie occupies one slot in the kernel's process table. The process table has a fixed maximum size. In long-running programs (like web servers) that fork many children without calling `wait()`, zombie accumulation can **exhaust the process table**, preventing ANY new processes from being created – a denial of service.

Example 5: Creating an Orphan Process

What This Example Teaches

An orphan process is created when the **parent exits first** while the child is still running. The init process (PID 1) automatically adopts the orphan. The child's `getppid()` will return 1 after adoption.

```

1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main() {
6     int pid = fork();
7
8     if (pid > 0) {
9         sleep(1);
10        printf("Parent exiting, PID = %d\n", getpid());
11    }
12
13    if (pid == 0) {
14        sleep(5);
15        printf("Child running, PID = %d\n", getpid());
16        printf("My parent PID is now: %d\n", getppid());
17    }
18
19    return 0;
20 }
```

Listing 6: orphan.c – Creating and observing an orphan process

Line-by-Line Explanation

```
int pid = fork();
```

Creates the child. `pid > 0` in parent, `pid == 0` in child. Notice: we use **two separate if statements** (not if/else). This is a common alternative style – both blocks are checked, but only one condition is true per process.

```
if (pid > 0) {
```

Parent block.

```
sleep(1);
```

Parent sleeps for 1 second. This gives the child time to start before the parent exits. The child will sleep for 5 seconds total (next block), so the parent's 1-second sleep ensures the parent exits FIRST.

```
printf("Parent exiting, PID = %d\n", getpid());
```

Parent announces it is about to exit.

After the `printf`, the parent reaches `return 0` and exits. At this moment, the child is still sleeping (only 1 second of its 5-second sleep has passed). The child is now an **orphan** – its parent is gone.

```
if (pid == 0) {
```

Child block. Note: uses `if`, not `else if`. When the parent evaluates this, `pid > 0` is true so this block is skipped. When the child evaluates this, `pid == 0` is true so this block runs.

```
sleep(5);
```

Child sleeps 5 seconds. During this time:

- First 1 second: both parent and child are sleeping
- After 1 second: parent wakes up, prints, and exits
- Seconds 1–5: child is an orphan, sleeping, adopted by `init`

```
printf("Child running, PID = %d\n", getpid());
```

After 5 seconds, child wakes and prints its own PID.

```
printf("My parent PID is now:  %d\n", getppid());
```

Key line! Calls `getppid()` to find the current parent's PID. Since the original parent exited, the `init` process (PID 1) adopted this child. This will print **1** – proving orphan adoption.

Expected Output

```
Parent exiting, PID = 4520    <-- parent prints after 1 second, then exits
Child running, PID = 4521    <-- child prints after 5 seconds
My parent PID is now: 1      <-- init adopted the orphan!
```

Orphan vs Zombie Summary

Zombie: Child exits first, parent never calls `wait()` $\Rightarrow$ child's record stays in process table as ghost.

Orphan: Parent exits first while child still runs $\Rightarrow$ `init` (PID 1) adopts child. Child is fine, just has a new parent.

Orphan is much less harmful than zombie. Orphans keep running normally under `init`.

Example 6: Simple exec()

What This Example Teaches

How `exec1()` completely **replaces** the current process image with a new program. After a successful `exec()`, the code that follows it in your file **never runs** – because your program is gone.

```

1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main() {
6     printf("About to run ls command...\n");
7
8     execl("/bin/ls", "ls", (char *)0);
9
10    perror("execl failed");
11    return 1;
12 }
```

Listing 7: `exec_simple.c` – Simple `exec()` replacing the process

Line-by-Line Explanation

```
printf("About to run ls command...\n");
```

This runs BEFORE `exec()`. The process still has its original code at this point. After this line, `exec()` will destroy this program.

```
execl("/bin/ls", "ls", (char *)0);
```

The `exec()` call. Breaking down the arguments:

Argument 1: `"/bin/ls"` – the **full filesystem path** to the program to run

Argument 2: `"ls"` – by convention, this is `argv[0]` of the new program (the program's name). This is what appears in `ps` output. Usually the same as the program name.

Argument 3: `(char *)0` – a NULL pointer marking the **end of arguments**. Required! Without this, `exec()` reads garbage from the stack as arguments.

What happens: The kernel loads `/bin/ls` into memory, replaces this process's code/data/stack with `ls`'s code/data/stack, and starts running `ls`. The PID stays the same. Your original code is gone.

```
perror("execl failed");
```

This line **only runs if `exec()` FAILED**. If `exec` succeeds, execution never returns here – the process is now running `ls`. If you see this error message, it means the path was wrong, the file doesn't exist, or there's a permissions issue.

```
return 1;
```

Returns error code 1 – also only reachable if exec() failed.

Expected Output

```
About to run ls command...  <-- our printf before exec
basic_fork.c zombie.c orphan.c exec_simple.c  <-- ls output
The "execl failed" line NEVER prints if exec() succeeds.
```

OS Concept – exec() Keeps the PID

exec() does NOT create a new process. It **replaces** the current process's program. The PID stays the same. Open file descriptors (like stdin, stdout, stderr) are also inherited by the new program. This is why fork() + exec() is the standard pattern: fork() creates a new PID, then exec() replaces that child with the desired program, leaving the parent untouched.

execl()	execlp()	execv()	Key Difference
execl("/bin/ls", "ls", NULL)	execlp("ls", "ls", NULL)	uses array	execlp searches PATH; execl needs full path

Example 7: fork() + exec() Together – Mini Shell

What This Example Teaches

The most important pattern in OS programming: **fork()** + **exec()** together. The parent forks a child, the child **exec()**s a new program, and the parent waits. This is exactly how real shells (bash, zsh) execute every command you type.

```

1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/wait.h>
7
8 int main() {
9     int PID;
10    char cmd[256];
11
12    printf("Press 'e' to quit. Type a command to execute.\n");
13
14    while (1) {
15        printf("cmd: ");
16        scanf("%s", cmd);
17
18        if (strcmp(cmd, "e") == 0)
19            exit(0);
20
21        PID = fork();
22
23        if (PID > 0) {
24            wait(NULL);
25        }
26        else if (PID == 0) {
27            execlp(cmd, cmd, NULL);
28
29            fprintf(stderr, "Cannot execute %s\n", cmd);
30            exit(1);
31        }
32        else {
33            fprintf(stderr, "Cannot create a new process\n");
34            exit(2);
35        }
36    }
37 }

```

Listing 8: mini_shell.c – fork() + exec() Together (Mini Shell)

Line-by-Line Explanation

```
#include <string.h>
```

Provides `strcmp()` for comparing strings. Needed to check if the user typed “e”.

```
#include <sys/wait.h>
```

Provides `wait()` system call. Needed to prevent zombie processes after each command execution.

```
int PID; char cmd[256];
```

Declares the variables:

PID – stores the return value of `fork()` (which child was created)

cmd[256] – a character array to hold the command the user types (up to 255 characters + null terminator)

```
printf("Press 'e' to quit. Type a command to execute.\n");
```

Welcome message. Tells the user the two possible inputs.

```
while (1) {
```

Infinite loop – keeps asking for commands until the user types “e”. Each iteration: prompt → read command → fork → exec → wait → repeat.

```
printf("cmd: "); scanf("%s", cmd);
```

Prints the prompt and reads one word (no spaces) from the user into `cmd`. `%s` reads until whitespace. The command name (e.g., “ls”, “pwd”, “date”) is stored in `cmd`.

```
if (strcmp(cmd, "e") == 0) exit(0);
```

Checks if the user typed “e” (exit). `strcmp()` returns 0 when the two strings are **equal**. If equal, call `exit(0)` to terminate the entire program cleanly.

```
PID = fork();
```

Creates a child process. Parent gets child’s PID in `PID`. Child gets 0.

```
if (PID > 0) { wait(NULL); }
```

Parent block: calls `wait(NULL)` to **block until the child finishes**. This serves two purposes:

1. Prevents zombie (reaps the child’s exit status)
 2. Makes the mini-shell wait for one command to finish before accepting the next
- After `wait()` returns, the parent loops back to print “cmd:” and wait for the next command.

```
else if (PID == 0) {
```

Child block. The child will replace itself with the requested command.

```
execlp(cmd, cmd, NULL);
```

Child replaces itself with the user's command.

`execlp` (note the “p”) searches the system's PATH directories for the command – you don't need the full path (`/bin/ls`), just the name (“ls”).

First cmd – path/name to find and execute

Second cmd – `argv[0]` of the new program (its own name)

NULL – end of argument list

If this line succeeds, the child is now running `ls` (or whatever was typed). The lines below never execute.

```
fprintf(stderr, "Cannot execute %s\n", cmd); exit(1);
```

Only reached if `execlp()` FAILED (command not found, no permissions, etc.).

`fprintf(stderr, ...)` writes the error message to standard error (FD 2) instead of stdout. Child exits with code 1 (error).

```
else { fprintf(stderr, "Cannot create a new process\n"); exit(2); }
```

`fork()` returned -1: failure. Print error to stderr and exit. Exit code 2 is used to distinguish this error from the exec failure (code 1).

Sample Session

Press 'e' to quit. Type a command to execute.

cmd: ls

basic_fork.c zombie.c orphan.c exec_simple.c mini_shell.c

cmd: pwd

/home/student/lab05

cmd: date

Sat May 10 14:23:45 PKT 2025

cmd: e

\$ <-- program exits cleanly

OS Concept – This is How Real Shells Work

When you type `ls` in bash, bash does exactly this:

1. Reads “ls” from your keyboard
2. Calls `fork()` – creates a child bash process
3. Child calls `execlp("ls", "ls", NULL)` – replaces itself with `ls`
4. Parent bash calls `wait()` – waits for `ls` to finish
5. `ls` prints its output and exits
6. Parent bash wakes up and shows the prompt again

Everything in Lab 05 (fork, exec, wait, PID, zombie prevention) is used in this one pattern.

Quick Reference – Quiz Cheat Sheet

fork() Rules – Must Know

- Code **before** fork() runs **once** (only in parent)
- Code **after** fork() runs **twice** (in both parent and child)
- fork() > 0 ⇒ you are the **parent** (value is child's PID)
- fork() == 0 ⇒ you are the **child**
- fork() < 0 ⇒ **error**, no child created
- Parent and child have **SEPARATE** address spaces after fork

exec() Rules – Must Know

- exec() **replaces** the process image – same PID, new program
- Code **after** exec() NEVER runs if exec succeeds
- Always end argument list with NULL or (char *)0
- execl() needs **full path**; execlp() searches **PATH**
- Always use fork() + exec() together so parent is not replaced

Zombie & Orphan Rules

- **Zombie:** Child exits first, parent never calls wait() ⇒ shows as <defunct>
- **Fix zombie:** Parent calls wait(NULL) after fork
- **Orphan:** Parent exits first ⇒ init (PID 1) adopts child
- Orphan's getppid() returns 1 after adoption

Compilation

```
gcc basic_fork.c      -o fork1          -Wall
gcc separate_addr.c  -o fork2          -Wall
gcc multi_fork.c     -o fork3          -Wall
gcc zombie.c         -o zombie         -Wall
gcc orphan.c         -o orphan         -Wall
gcc exec_simple.c    -o exec1          -Wall
gcc mini_shell.c     -o mini_shell     -Wall
```

Listing 9: Compilation commands for Lab 05

Common Mistakes – Watch Out!

1. **Missing (char *)0 in execl():** exec reads garbage from stack and crashes.
2. **No wait() in parent:** Every forked child becomes a zombie.
3. **No exit() in child after exec fails:** Child falls through to parent's code.

4. **Calling fork() after exec():** exec replaces the program – fork() never runs.
5. **Confusing fork() return values:** Parent gets child PID (> 0), child gets 0.

Good luck on your quiz!

Key pattern: `fork()` → check return → child: `exec()` → parent: `wait()`